ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Nguyễn Vũ Đức Anh

BÀI TẬP 6 – LẬP TRÌNH THUẬT TOÁN LẬP LỊCH CPU HÀNG ĐỢI ĐA CẤP

Ngành: Mạng máy tính và truyền thông dữ liệu

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

HÀ NỘI - 2026

Nguyễn Vũ Đức Anh

BÀI TẬP 6 – LẬP TRÌNH THUẬT TOÁN LẬP LỊCH CPU HÀNG ĐỢI ĐA CẤP

Ngành: Mạng máy tính và truyền thông dữ liệu

Supervisor:
Co-Supervisor:

HÀ NỘI - 2026

TÓM TẮT

Tóm tắt: Lập trình thuật toán lập lịch CPU hang đợi đa cấp bằng java. Hệ thống gồm 3 hàng đợi: hàng đợi 1 sử dụng thuật toán round robin, hàng đợi 2 sử dụng thuật toán SJF cho phép dừng, hàng đợi 3 sử dụng thuật toán FCFS. Nhận input vào là file .txt và output in ra thông tin của từng tiến trình và trung bình của từng tiến trình.

I. Class Process:

Class Process để tạo đối tượng Process bao gồm các thuộc tính ID, thời gian chạy, thời gian đến, thời gian còn lại, hàng đợi được thêm vào, thời gian chờ, thời gian phản hồi, thời gian hoàn thành.

Chuỗi hằng IDLE_ID dùng để thể hiện trạng thái CPU khi không có tiến trình nào được chạy.

```java
public class Process implements Comparable<Process> {  26 usages
    private String processID;  3 usages
    private int burstTime;  2 usages
    private int arrivalTime;  2 usages
    private int remainTime;  5 usages
    private String readyQueue;  2 usages
    private int waitTime = 0;  3 usages
    private int responseTime = 0;  3 usages
    private int turnAroundTime = 0;  3 usages

    public static final String IDLE_ID = "IDLE";  4 usages

    public Process (String processID, int burstTime, int arrivalTime, String readyQueue) {  1 usage
        this.processID = processID;
        this.burstTime = burstTime;
        this.arrivalTime = arrivalTime;
        this.readyQueue = readyQueue;
        this.remainTime = burstTime;
    }
```

Các hàm get/set để thay đổi giá trị các thuộc tính.

```java
    public String getProcessID() { return processID; }

    public int getBurstTime() { return burstTime; }

    public int getArrivalTime() { return arrivalTime; }

    public int getRemainTime() { return remainTime; }

    public String getReadyQueue() { return readyQueue; }

    public void minusRemainTime() { this.remainTime--; }

    public int getWaitTime() { return waitTime; }

    public void plusWaitTime() { this.waitTime++; }

    public int getResponseTime() { return responseTime; }

    public void setResponseTime(int responseTime) { this.responseTime = responseTime; }

    public int getTurnAroundTime() { return turnAroundTime; }

    public void setTurnAroundTime(int turnAroundTime) { this.turnAroundTime = turnAroundTime; }
```

Class triển khai Comparable và ghi đè compareTo để có thể so sánh giữa các Process với nhau, hàm printProcess để in ra thông tin process.

```java
    @Override
    public int compareTo(Process o) { return Integer.compare(this.remainTime, o.remainTime); }

    public void printProcess () {  1 usage
        String s = "Process ID: " + processID
                    + ", wait time: " + waitTime
                    + ", response time: " + responseTime
                    + ", turnaround time: " + turnAroundTime;
        System.out.println(s);
    }
}
```

II. Class Dispatcher:

Class Dispatcher tạo bộ điều phối bao gồm:

- 2 hàng đợi để làm ready queue cho FCFS và round robin
- 1 piority queue cho SJF
- 1 đối tượng current thể hiện tiến trình đang được chạy hiện tại
- biến đếm contentSwitch đếm số lần đổi tiến trình
- biến lastID lưu ID của tiến trình hiện tại (dùng để đếm contentSwitch), khi chưa có tiến trình nào thì là IDLE_ID
- chuỗi hằng FCFS, PSJF và RR là tên 3 hàng đợi

```java
public class Dispatcher {  7 usages
    private final PriorityQueue<Process> PreemptiveSJFReadyQueue = new PriorityQueue<>();  10 usages
    private final Queue<Process> FCFSReadyQueue = new ArrayDeque<>();  5 usages
    private final Queue<Process> RobinReadyQueue = new ArrayDeque<>();  9 usages

    private Process current = null;  27 usages
    private int contextSwitch = 0;  2 usages
    private String lastID = Process.IDLE_ID;  4 usages

    private static final int timeQuantum = 20;  1 usage
    public static final String FCFS = "FCFS";  2 usages
    public static final String PSJF = "PSJF";  3 usages
    public static final String RR = "RR";  2 usages
```

Các hàm để thêm tiến trình vào các hang đợi, getter CPUswitch, hàm isBusy kiểm tra xem readyQueue và CPU hiện tại có tiến trình nào không.

```java
    public void addFCFSReadyQueue(Process process) { FCFSReadyQueue.add(process); }

    public void addPreemptiveSJFReadyQueue(Process process) { PreemptiveSJFReadyQueue.add(process); }

    public void addRobinReadyQueue (Process process) { RobinReadyQueue.add(process); }

    public int getContextSwitch() { return contextSwitch; }

    public boolean isBusy() {  1 usage
        return !FCFSReadyQueue.isEmpty() || !PreemptiveSJFReadyQueue.isEmpty() ||
                !RobinReadyQueue.isEmpty() || current != null;
    }
```

Hàm changeContext nhận tham số thời gian, thực hiện tính response time cho lần đầu tiến trình được xử lý, đếm số lần đổi tiến trình và cập nhật lastID.

```java
private void changeContext(int time) {  6 usages
    if (current.getRemainTime() == current.getBurstTime()) {
        current.setResponseTime(time - current.getArrivalTime());
    }

    if (!lastID.equals(Process.IDLE_ID)) {
        contextSwitch++;
    }
    lastID = current.getProcessID();
}
```

Hàm executeCurrent nhận tham số thời gian, trả về process được hoàn thành. Thực hiện trừ thời gian thực thi còn lại của process, tăng thời gian chờ các process ở các hang đợi. Nếu process current hoàn thành, tính turn around time và trả về process.

```java
private Process executeCurrent(int time) {  1 usage
    current.minusRemainTime();

    for (Process process : RobinReadyQueue) {
        process.plusWaitTime();
    }

    for (Process process : PreemptiveSJFReadyQueue) {
        process.plusWaitTime();
    }

    for (Process process : FCFSReadyQueue) {
        process.plusWaitTime();
    }

    if (current.getRemainTime() == 0) {
        current.setTurnAroundTime(time + 1 - current.getArrivalTime());
        Process finished = current;
        current = null;
        return finished;
    }
    return null;
}
```

Hàm schedule nhận tham số thời gian, trả về process được hoàn thành. Thực hiện kiểm tra current hiện tại null không. Nếu có thì lần lượt kiểm tra theo trình tự các hang đợi round robin, PreemptiveSJF, FCFS để cho vào current. Trả về null nếu tất cả hang chờ trống.

```java
public Process Schedule (int time){  1 usage
    if (current == null){
        if (!RobinReadyQueue.isEmpty()) {
            current = RobinReadyQueue.poll();
            changeContext(time);
        } else if (!PreemptiveSJFReadyQueue.isEmpty()) {
            current = PreemptiveSJFReadyQueue.poll();
            changeContext(time);
        } else if (!FCFSReadyQueue.isEmpty()){
            current = FCFSReadyQueue.poll();
            changeContext(time);
        } else {
            if (!lastID.equals(Process.IDLE_ID)){
                lastID = Process.IDLE_ID;
            }
            return null;
        }
    } else {
```

Nếu current không rỗng, thực hiện kiểm tra xem current thuộc hang chờ nào. Nếu thuộc về PreemptiveSJF thì kiểm tra xem có tiến trình nào có ưu tiên cao hơn không để chạy. Nếu thuộc về round robin thì kiểm tra đã chạy hết quantum time để đổi tiến trình. Trả về kết quả của hàm executeCurrent để thực thi tiến trình hiện tại.

```java
    } else {
        if (current.getReadyQueue().equals(PSJF)){
            if (!RobinReadyQueue.isEmpty()) {
                PreemptiveSJFReadyQueue.add(current);
                current = RobinReadyQueue.poll();
                changeContext(time);
            } else if (!PreemptiveSJFReadyQueue.isEmpty() &&
                    PreemptiveSJFReadyQueue.peek().getRemainTime() < current.getRemainTime()) {
                PreemptiveSJFReadyQueue.add(current);
                current = PreemptiveSJFReadyQueue.poll();
                changeContext(time);
            }
        } else if (current.getReadyQueue().equals(RR)){
            if ((current.getBurstTime() - current.getRemainTime())%timeQuantum == 0){
                RobinReadyQueue.add(current);
                current = RobinReadyQueue.poll();
                changeContext(time);
            }
        }
    }

    return executeCurrent(time);
}
```

III. Class SystemOS:

Class Dispatcher quản lý các Process cần chạy, process hoàn thành và bộ điều phối, thực hiện đếm thời gian và khởi tạo bộ điều phối.

```java
public class SystemOS {  2 usages
    private int time = 0;  4 usages
    private final ArrayList<Process> jobQueue = new ArrayList<>();  3 usages
    private final ArrayList<Process> finishQueue = new ArrayList<>();  3 usages
    private final Dispatcher dispatcher;  7 usages

    public SystemOS () { dispatcher = new Dispatcher(); }
```

Hàm readFile thực hiện đọc file từ đường dẫn, kiểm tra file tồn tại không, định dạng, giá trị đầu vào hợp lệ.

```java
    private void readFile (String filePath) {  1 usage
        try {
            File inputFile = new File (filePath);
            Scanner reader = new Scanner(inputFile);

            while (reader.hasNext()) {
                String line = reader.nextLine().trim();
                if (line.isEmpty()) continue;

                String[] parts = line.split( regex: "\\s+");

                if (parts.length != 4) {
                    System.out.println("Invalid input format!");
                    System.exit( status: 1);
                }

                try {
                    String processID = parts[0];

                    int burstTime = Integer.parseInt(parts[1]);
                    int arrivalTime = Integer.parseInt(parts[2]);
                    String readyQueue = parts[3];

                    if ((!readyQueue.equals(Dispatcher.FCFS) && !readyQueue.equals(Dispatcher.PSJF)
                            && !readyQueue.equals(Dispatcher.RR))
                        || burstTime <= 0 || arrivalTime < 0) {
                        System.out.println("Invalid input value!");
                        System.exit( status: 1);
                    }

                    jobQueue.add(new Process(processID, burstTime, arrivalTime, readyQueue));

                } catch (NumberFormatException e) {
                    System.out.println("Invalid input format!");
                    System.exit( status: 1);
                }
            }
            reader.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found!");
            System.exit( status: 1);
        }
    }
```

Hàm printResult thực hiện tính thời gian chờ trung bình, thời gian phản hồi trung bình, thời gian hoàn thành trung bình. In ra kết quả từng process và kết quả cả quá trình.

```java
private void printResult (){  1 usage
    double meanWaitTime = 0;
    double meanResponseTime = 0;
    double meanTurnAroundTime = 0;
    double numProcess = finishQueue.size();

    for(Process process : finishQueue) {
        process.printProcess();
        meanWaitTime += process.getWaitTime();
        meanResponseTime += process.getResponseTime();
        meanTurnAroundTime += process.getTurnAroundTime();
    }

    System.out.println("Mean wait time: " + meanWaitTime/numProcess
                       + ", Mean response time: " + meanResponseTime/numProcess
                       + ", Mean turn around time: " + meanTurnAroundTime/numProcess
                       + ", context switch: " + dispatcher.getContextSwitch());
}
```

Hàm Run nhận đường dẫn file và truyền cho hàm readFile. Thực hiện đến khi jobqueue trống và dispatcher hết bận: chuyển process từ jobqueue vào readyqueue tại thời điểm arrival, thực hiện lập lịch Schedule, lưu kết quả vào finishQueue và tang biến đếm thời gian. Thực hiện in ra kết quả khi hoàn thành.

```java
public void Run (String inputPath) {  1 usage
    readFile(inputPath);

    while (!jobQueue.isEmpty() || dispatcher.isBusy()) {
        Iterator<Process> it = jobQueue.iterator();

        while (it.hasNext()) {
            Process process = it.next();
            if (process.getArrivalTime() <= time) {
                if (process.getReadyQueue().equals(Dispatcher.FCFS)) {
                    dispatcher.addFCFSReadyQueue(process);
                } else if (process.getReadyQueue().equals(Dispatcher.PSJF)) {
                    dispatcher.addPreemptiveSJFReadyQueue(process);
                } else {
                    dispatcher.addRobinReadyQueue(process);
                }
                it.remove();
            }
        }

        Process finished = dispatcher.Schedule(time);

        if (finished != null) {
            finishQueue.add(finished);
        }

        time++;
    }
    printResult();
}
```

IV. Class Main:

Nơi bắt đầu chương trình, khởi tạo SystemOS và truyền vào đường dẫn tới file input.

```java
public class Main {
    public static void main (String[] args) {
        SystemOS system1 = new SystemOS();
        system1.Run( inputPath: "src/input.txt");
    }
}
```

V. Chạy và kết quả:

Testcase 1:

Main.java | Process.java | Dispatcher.java | SystemOS.java | input.txt

```
P1 0 0 FCFS
```

```
Invalid input value!

Process finished with exit code 1
```

Testcase 2:

Main.java | Process.java | Dispatcher.java | SystemOS.java | input.txt

```
P1 7 0 FCFS
P2
```

```
Invalid input format!

Process finished with exit code 1
```

Testcase 3:

Main.java | Process.java | Dispatcher.java | SystemOS.java | input.txt

```
P1 50 0 RR
P2 15 30 RR
P3 45 30 RR
P4 40 0 PSJF
P5 10 120 PSJF
P6 30 60 FCFS
P7 20 130 FCFS
```

```
Process ID: P2, wait time: 10, response time: 10, turnaround time: 25
Process ID: P1, wait time: 35, response time: 0, turnaround time: 85
Process ID: P3, wait time: 35, response time: 25, turnaround time: 80
Process ID: P5, wait time: 0, response time: 0, turnaround time: 10
Process ID: P4, wait time: 120, response time: 110, turnaround time: 160
Process ID: P6, wait time: 100, response time: 100, turnaround time: 130
Process ID: P7, wait time: 60, response time: 60, turnaround time: 80
Mean wait time: 51.42857142857143, Mean response time: 43.57142857142857, Mean turn around time: 81.42857142857143, context switch: 11

Process finished with exit code 0
```